

Inheritance

- **Inheritance allows you to create a general class and then define specialized classes that have access to the members of the general class.**
- **These new specialized classes can extend functionality by adding their own new unique data and behaviors.**
- Inheritance is associated with an “is a” relationship.
- A specialized class “is a” form of the general class

Example

- you might create a **class called Person** and include **data characteristics of identification** number, name, age, and home address.
- In regions that use a Social Security number for identification such as here in the United States, the Social Security number would be the identification number.
- Behaviors or methods titled Eat(), Walk(), and Talk() might be included.
- After these are defined, you could create specialized classes such as Student or Faculty that inherit from the Person **class**.
- **Student “is a” Person, just as Faculty “is a” Person** Student might have unique data characteristics of student ID, major, and year in school.

Inheriting from Other .NET FCL Classes

- The Windows forms classes that is created inherit from the `System.Windows.Forms.Form` **class**.
- **Adding** instantiated objects from classes such as `Button`, `Label`, `TextBox`, and `ListBox` enabled you to add functionality to your program with minimal programming.
- To inherit from the `System.Windows.Forms.Form` **class**, **new entries were added to your class definition.**

- Through using inheritance, applications instantiating objects of the Student **class** **would have not only the unique characteristics of the student (student ID, major, and year)**, but also have access to the characteristics of the Person (identification number, name, age, and address).

- Classes can also have a “has a” relationship in which a single **class is defined to have** instances of other **class types**.
- **This is a concept called containment or aggregation.**
- **You** experience containment in C# when you define classes that instantiate objects of the **string or int classes**.
- **However, “has a” relationships are usually associated with userdefined classes.**
- For example, a Person “has a” medicalRecord and “has a” dentalRecord.

Inheriting from the Object Class

- Every object created in C# automatically inherits from a base class named **object** that is in the **System** namespace.
- **Object** has four methods that every class written in C# inherits as long as it provides a reference to the **System** namespace.
- One of these, the **ToString()** method, you have used in many of your programs.

```
Public class Person
{
Private string idNumber;
private string lastName;
Private string firstName;
Private int age;
Public Person()
// Constructor with zero arguments
{
idNumber="";
lastName="unknown";
firstName=string.Empty;
age=0;
}
Public Person(string id, string lname,string fname, int anAge)
//Constructorwith fourfarguments
{
idNumber =id;
lastName=lname;
firstName=fname;
age=anAge;
}
```



```
Public Person(string id, string lname, string fname)
```

```
{//Constructor with three arguments
```

```
idNumber=id;
```

```
lastName=lname;
```

```
firstName=fname;
```

```
age=0;
```

```
}
```

```
Public Person(string id)
```

```
{//Constructor with one argument
```

```
idNumber=id;
```

```
lastName="unknown";
```

```
firstName="unknown";
```

```
Age=0;
```

```
}
```

```
}
```

- Inheritance, together with encapsulation and polymorphism, is one of the three primary characteristics (or pillars) of object-oriented programming.
- Inheritance enables you to create new classes that reuse, extend, and modify the behavior that is defined in other classes.
- The class whose members are inherited is called the base class, and the class that inherits those members is called the derived class.
- A derived class can have only one direct base class.
- However, inheritance is transitive.
- If ClassC is derived from ClassB, and ClassB is derived from ClassA, ClassC inherits the members declared in ClassB and ClassA.

- Conceptually, a derived class is a specialization of the base class.
- For example, if you have a base class Animal, you might have one derived class that is named Mammal and another derived class that is named Reptile.
- A Mammal is an Animal, and a Reptile is an Animal, but each derived class represents different specializations of the base class.

- When you define a class to derive from another class, the derived class implicitly gains all the members of the base class, except for its constructors and destructors.
- The derived class can thereby reuse the code in the base class without having to re-implement it.
- In the derived class, you can add more members.
- In this manner, the derived class extends the functionality of the base class

using System;

```
public class Parent
{
    string parentString;
    public Parent()
    {
        Console.WriteLine("Parent Constructor.");
    }
    public Parent(string myString)
    {
        parentString = myString;
        Console.WriteLine(parentString);
    }
    public void print()
    {
        Console.WriteLine("I'm a Parent Class.");
    }
}
```

```
public class Child : Parent
{
    public Child() : base("From Derived")
    {
        Console.WriteLine("Child
        Constructor.");
    }
    public new void print()
    {
        base.print();
        Console.WriteLine("I'm a Child Class.");
    }
    public static void Main()
    {
        Child child = new Child();
        child.print();
        ((Parent)child).print();
    }
}
```

Output:

From Derived Child Constructor. I'm a Parent
Class. I'm a Child Class. I'm a Parent Class.

Example – Inherit1

- Like all classes, it derives from [System.Object](#) and inherits all its methods.
- **System.Object is the ultimate base class of all classes in the .NET Framework**

Object.Equals Method (Object)

- Determines whether the specified object is equal to the current object.
- Return type : true if the specified object is equal to the current object; otherwise, false

Eg., : `Console.WriteLine("Calling Equals:");`
`Console.WriteLine("person1a and person1b: {0}", person1a.Equals(person1b));`
`Console.WriteLine("person1a and person2: {0}", person1a.Equals(person2));`

Output :

person1a and person1b: True
person1a and person2: False

Object.GetHashCode Method

- A hash code is a numeric value that is used to insert and identify an object in a hash-based collection such as the
 - [Dictionary<TKey, TValue>](#) class,
 - the [Hashtable](#) class,
 - or a type derived from the [DictionaryBase](#) class.
- The GetHashCode method provides this hash code for algorithms that need quick checks of object equality.

Object.GetType Method

- Gets the [Type](#) of the current instance.
- For two objects x and y that have identical runtime types,
`Object.ReferenceEquals(x.GetType(),y.GetType())`
returns true.
- The following example uses the GetType method with the [ReferenceEquals](#) method to determine whether one numeric value is the same type as two other numeric values.

Example

```
int n1 = 12;
```

```
int n2 = 82;
```

```
long n3 = 12;
```

```
Console.WriteLine("n1 and n2 are the same type: {0}",  
    Object.ReferenceEquals(n1.GetType(), n2.GetType()));
```

```
Console.WriteLine("n1 and n3 are the same type: {0}",  
    Object.ReferenceEquals(n1.GetType(), n3.GetType()));
```

The example displays the following output:

n1 and n2 are the same type: True

n1 and n3 are the same type: False

Object.ToString Method

- Returns a string that represents the current object.
- ToString is the major formatting method in the .NET Framework.
- It converts an object to its string representation so that it is suitable for display.

```
using System;  
public class Example {  
    public static void Main()  
    {  
        Object obj = new Object();  
        Console.WriteLine(obj.ToString());  
    }  
}
```

```
// The example displays the following output: //  
System.Object
```

Object.MemberwiseClone Method

- Creates a shallow copy of the current [Object](#).
- For example,
 - consider an object called X that references objects A and B.
 - Object B, in turn, references object C.
 - A shallow copy of X creates new object X2 that also references objects A and B.
 - In contrast, a deep copy of X creates a new object X2 that references the new objects A2 and B2, which are copies of A and B.
 - B2, in turn, references the new object C2, which is a copy of C.

Object.ReferenceEquals Method

- Determines whether the specified [Object](#) instances are the same instance.

- Syntax

```
public static bool ReferenceEquals(  
    Object objA,  
    Object objB  
)
```

- **Parameters**

- objA Type: [System.Object](#)
The first object to compare.
- objB Type: [System.Object](#)
The second object to compare.

- **Return Value**

- Type: [System.Boolean](#)
true if objA is the same instance as objB or if both are null; otherwise, false.

Employee class

Implementing Inheritance

Accessibility

- All types and type members have an accessibility level, which controls whether they can be used from other code in your assembly or other assemblies.

- You can use the following access modifiers to specify the accessibility of a type or member when you declare it:
- public The type or member can be accessed by any other code in the same assembly or another assembly that references it.
- private The type or member can be accessed only by code in the same class or struct.

- protected The type or member can be accessed only by code in the same class or struct, or in a class that is derived from that class.
- internal The type or member can be accessed by any code in the same assembly, but not from another assembly.

- **protected internal** The type or member can be accessed by any code in the assembly in which it is declared, or from within a derived class in another assembly.
- Access from another assembly must take place within a class declaration that derives from the class in which the protected internal element is declared, and it must take place through an instance of the derived class type.

- The following example demonstrate how to specify access modifiers on a type and member:

```
public class Bicycle {  
    public void Pedal() { }  
}
```
- Not all access modifiers can be used by all types or members in all contexts, and in some cases the accessibility of a type member is constrained by the accessibility of its containing type.

Overriding Inherited behavior

- A derived class can override behavior inherited from the base class, which allows a derived type to apply more specific behavior

Virtual and Override Keywords

- The Virtual keyword indicates that a member is a candidate for being overridden in a derived class
- Virtual can be applied to methods, properties, indexes and events
- Virtual and override keywords complement each other
- The virtual keyword propagates to all descendants
- This means that the virtual method is overridable in not just the most immediate derived class, but in all descendants.

- Only if a method is declared virtual, derived classes can override this method if they are explicitly declared to override the virtual base class method with the **override** keyword.

```
using System;
namespace Override
{
    class A
    {
        public virtual void Vtest() { Console.WriteLine("A::Vtest()"); }
    }

    class B : A
    {
        public override void Vtest() { Console.WriteLine("B::Vtest()"); }
    }

    class Test
    {
        static void Main(string[] args)
        {
            A a;
            B b;

            a = new A();
            b = new B();
            a.Vtest(); // output --> "A::Vtest()"
            b.Vtest(); // output --> "B::Vtest()"

            a = new B();
            a.Vtest(); // output --> "B::Vtest()"
        }
    }
}
```

Overload versus Override

- Overload and Override are different concepts
- When a member of a base class is overridden, the signature of the base and the signature of the derived member are identical.
- Overloading requires different signatures
- In a derived class, a function can be overloaded, overridden or both

- Overloading means declaring methods with same name but different signatures because of this different tasks with same method name are performed.

- **Overloading** (Called Early Binding or Compile Time Polymorphism or static binding)
- **Overriding** (Called Late Binding or Run Time Polymorphism or dynamic binding)
-

- Method overriding or run time polymorphism means same method names with same signatures.

Extension Method

- We write C# programs that are used constantly.
- Some written yourself and many more that have been written by others.
- Sometimes you might wish a class had an additional method that would be useful to you.
- If original class is created, there are two options:
 - Revise the existing class, including the new useful method.
 - Derive a child class from the existing class and provide it with a new method.

- The classes created by others might not allow to either revise or extend them.
- But can create an entirely new class that includes your new method.
- But that would duplicate a lot of the work already done when the first class was created.
- The best option is to write an extension method.

- **Extension methods are methods you can write to add to any type.**

Example

- Int32 class is used to declare integers.
- Suppose a company that frequently uses customer account numbers,
- The company decides to add an extra digit to each account number.
- For simplicity, assume that all account numbers are two digits and that the new, third digit should be the rightmost digit in the sum of the first two digits.
- This problem can be handled this by creating a class named AccountNumber, including a method to produce the extra digit, and redefining every instance of a customer's account number in the application as an AccountNumber object.
- However, if there are already many applications that define the account number as an integer, you might prefer to create an extension method that extends the Int32 class.

Example

```
public static int AddCheckDigit(this int num)
{
    int first = num / 10;
    int second = num % 10;
    int third = (first + second) % 10;
    int result = num * 10 + third;
    return result;
}
```

- The above code contains a method that extends the Int32 class.
- The first parameter in an extension method specifies the type extended and must begin with the keyword **this**.
- For example, the first (and in this case, the only) parameter in the AddCheckDigit() method is int num

- Extension methods must be static methods.
- Within the `AddCheckDigit()` method , the first digit is extracted from the two-digit account number by dividing by 10 and taking the resulting whole number
- The second digit is extracted by taking the remainder.
- Those two digits are added, and the last digit of that sum is returned from the method.
- For example, if 49 is passed into the method, first becomes 4, second becomes 9, and third becomes the last digit of 13, or 3.
- Then the original number (49) is multiplied by 10 and added to the third digit, resulting in 493.

The *new* Modifier

- When used as a declaration modifier, the new keyword explicitly hides a member that is inherited from a base class.
- When you hide an inherited member, the derived version of the member replaces the base class version.
- Although you can hide members without using the new modifier, you get a compiler warning.
- If you use new to explicitly hide a member, it suppresses this warning.
- To hide an inherited member, declare it in the derived class by using the same member name, and modify it with the new keyword.

Example

```
public class BaseC
{
    public int x;
    public void Invoke() { }
}

public class DerivedC : BaseC
{
    new public void Invoke() { }
}
```

- BaseC.Invoke is hidden by DerivedC.Invoke.
- The field x is not affected because it is not hidden by a similar name

Abstract Classes

- Classes can be declared as abstract by putting the keyword `abstract` before the class definition.
 - `public abstract class A { // Class members here. }`
- An abstract class cannot be instantiated.
- The purpose of an abstract class is to provide a common definition of a base class that multiple derived classes can share.
 - For example, a class library may define an abstract class that is used as a parameter to many of its functions, and require programmers using that library to provide their own implementation of the class by creating a derived class.

- Abstract classes may also define abstract methods.
- This is accomplished by adding the keyword `abstract` before the return type of the method.
 - For example:

```
public abstract class A
{
    public abstract void DoWork(int i);
}
```


- Abstract methods have no implementation, so the method definition is followed by a semicolon instead of a normal method block.
- Derived classes of the abstract class must implement all abstract methods.
- When an abstract class inherits a virtual method from a base class, the abstract class can override the virtual method with an abstract method.

— For example:

```
public class D
{
    public virtual void DoWork(int i)
    {
        // Original implementation.
    }
}

public abstract class E : D
{
    public abstract override void DoWork(int i);
}

public class F : E
{
    public
    void DoWork(int i)
    { // New implementation.
    }
}
```

- If a virtual method is declared abstract, it is still virtual to any class inheriting from the abstract class.
- A class inheriting an abstract method cannot access the original implementation of the method
 - in the previous example, DoWork on class F cannot call DoWork on class D.
 - In this way, an abstract class can force derived classes to provide new method implementations for virtual methods.

Sealed Classes

- Sealed classes are the reverse of abstract classes.
- While abstract classes are inherited and refined in the derived type, sealed classes cannot be inherited and cannot be refined any further.
- Classes can be declared as sealed by putting the keyword sealed before the class definition.
 - For example:

```
public sealed class D  
{ // Class members here. }
```

- A sealed class cannot be used as a base class.
- For this reason, it cannot also be an abstract class.
- Sealed classes prevent derivation.
- A method, indexer, property, or event, on a derived class that is overriding a virtual member of the base class can declare that member as sealed.
- This negates the virtual aspect of the member for any further derived class.
- This is accomplished by putting the sealed keyword before the override keyword in the class member declaration.

– For example:

```
public class D : C
{
    public sealed override void DoWork() { }
}
```

Constructors and Destructors

- When any object is created, a constructor that has the same name as the class itself is called.

– Example:

```
SomeClass anObject = new SomeClass();
```

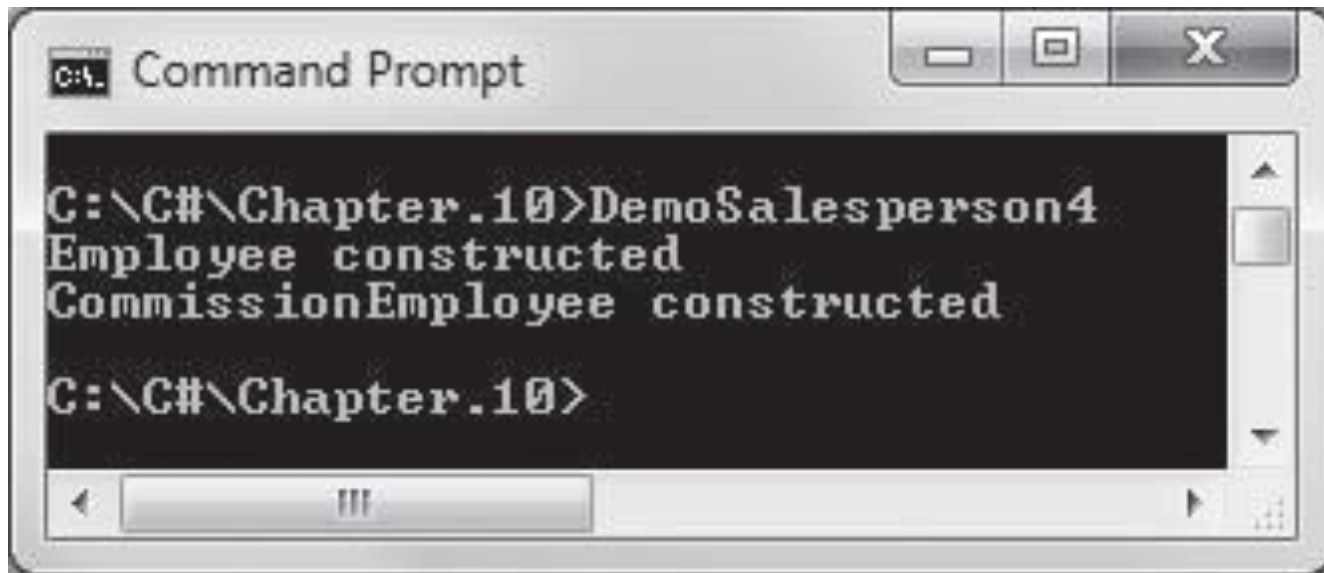
- When an object that is a member of a derived class is instantiated, both the constructor for the base class and the constructor for the extended, derived class are called.
- When any derived class object is created, the base class constructor must execute first; only then does the derived class constructor execute.

- In the earlier examples of inheritance ,
 - each class contained default constructors, so their execution was transparent.
- When a subclass instance is created , both the base and derived constructors execute.

```
– For example,  
class Employee  
{  
    private int empNum;  
    protected double empSal;  
    public Employee()  
    {  
        Console.WriteLine("Employee constructed");  
    }  
}  
class CommissionEmployee : Employee  
{  
    private double commissionRate;  
    public CommissionEmployee()  
    {  
        Console.WriteLine("CommissionEmployee constructed");  
    }  
}  
using System;  
public class DemoSalesperson4  
{  
    public static void Main()  
    {  
        CommissionEmployee salesperson = new CommissionEmployee();  
    }  
}
```

- Consider the abbreviated Employee and CommissionEmployee classes in the previous example.
 - Employee contains just two fields and a constructor;
 - CommissionEmployee descends from Employee and contains a constructor as well.

- The main program contains just one statement;
 - it instantiates a `CommissionEmployee`.



```
C:\C#\Chapter.10>DemoSalesperson4
Employee constructed
CommissionEmployee constructed
C:\C#\Chapter.10>
```

The output shows that this one statement causes both constructors to execute.

Interfaces

- Some object-oriented programming languages, notably C++, allow a subclass to inherit from more than one parent class.
- Example
 - Creation of an Employee class that contains data fields pertaining to each employee in an organization.
 - Creation of a Product class that holds information about each product an organization manufactures.
- When a Patent class is created for each product for which the company holds a patent, then include product information as well as information about the employee who was responsible for the invention.
- In this situation, it would be convenient to inherit fields and methods from both the Product and Employee classes.
- The ability to inherit from more than one class is called **multiple inheritance**.

- Multiple inheritance is a difficult concept, and programmers encounter many problems when they use it.
 - For example, variables and methods in the parent classes may have identical names, creating a conflict when the child class uses one of the names.
- Additionally, a child class constructor must call its parent class constructor.
- When two or more parents exist, this becomes a more complicated task:
 - To which class should base refer when a child class has multiple parents?

- For all of these reasons, multiple inheritance is prohibited in C#.
- However, C# does provide an alternative to multiple inheritance, known as an **interface**.
- Much like an abstract class, an interface is a collection of methods that can be used by any class as long as the class provides a definition to override the interface's abstract definitions.
- Within an abstract class, some methods can be abstract, while others need not be.
- Within an interface, all methods are abstract.

- An interface can be created much as an abstract class definition is created, except that the keyword **interface** instead of abstract class.
 - For example,
public interface IWorkable
{
string Work();
}

- The IWorkable interface contains a single method named Work().
- When any class implements IWorkable, it must also include a Work() method that returns a string.

```
class Employee : IWorkable
{
public Employee(string name)
{
Name = name;
}
public string Name {get; set;}
public string Work()
{
return "I do my job";
}
}
abstract class Animal : IWorkable
{
public Animal(string name)
{
Name = name;
}
public string Name {get; set;}
public abstract string Work();
}
```

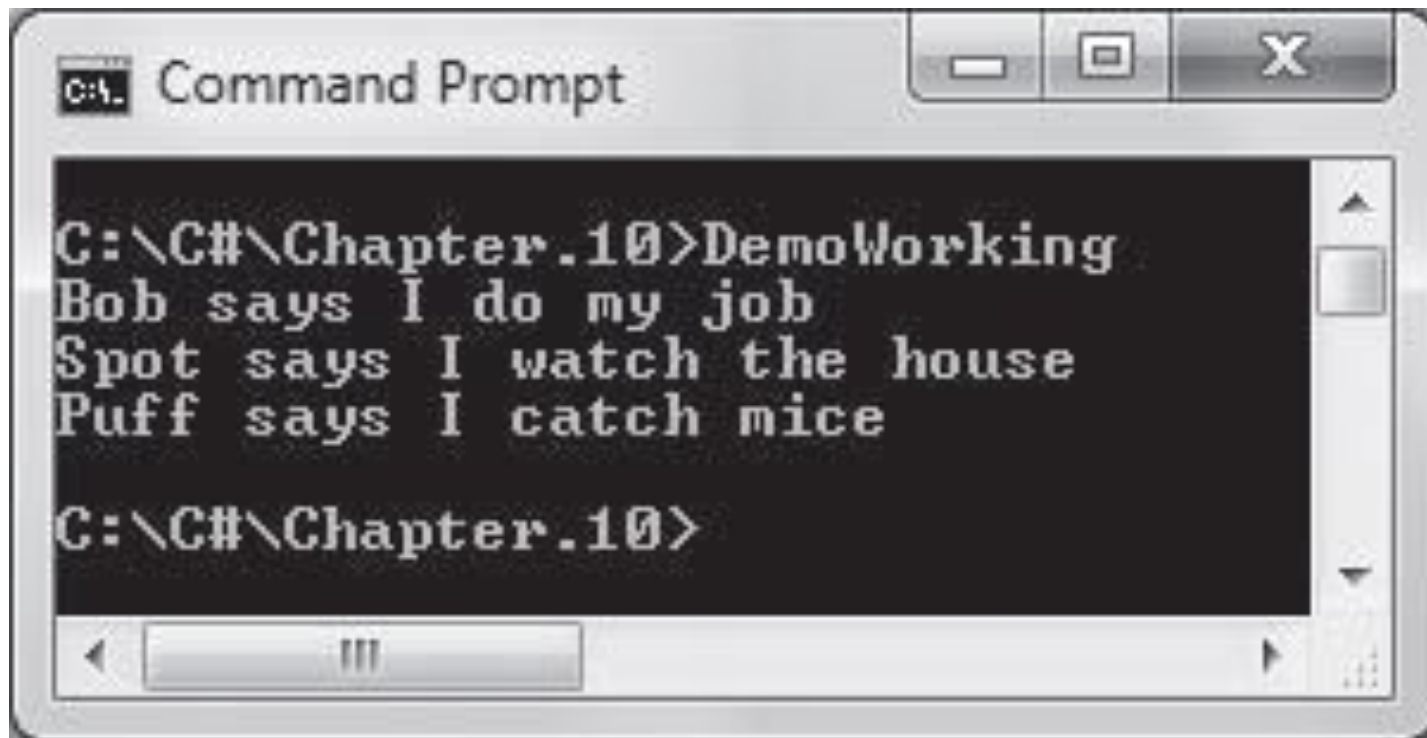
```
class Dog : Animal  
{  
public Dog(string name) : base(name)  
{  
}  
public override string Work()  
{  
return "I watch the house";  
}  
}  
class Cat : Animal  
{  
public Cat(string name) : base(name)  
{  
}  
public override string Work()  
{  
return "I catch mice";  
}  
}
```


- Example shows two classes that implement IWorkable:
 - the Employee class and the Animal class.
- Because each implements IWorkable, each must declare a Work() method.
- The Employee class implements Work() to return the “I do my job” string.
- The abstract Animal class defines Work() as an abstract method, meaning that descendants of Animal must implement Work().

```
using System;
class DemoWorking
{
    public static void Main()
    {
        Employee bob = new Employee("Bob");
        Dog spot = new Dog("Spot");
        Cat puff = new Cat("Puff");
        Console.WriteLine(bob.Name + " says " + bob.Work());
        Console.WriteLine(spot.Name + " says " + spot.Work());
        Console.WriteLine(puff.Name + " says " + puff.Work());
    }
}
```

- Note how `Work()` is defined differently for each.
- When you create a program that instantiates an `Employee`, a `Dog`, or a `Cat`, as in the previous `DemoWorking` program, each object type knows how to “`Work()`” appropriately.

Output



```
C:\C#\Chapter.10>DemoWorking
Bob says I do my job
Spot says I watch the house
Puff says I catch mice

C:\C#\Chapter.10>
```

- Abstract classes and interfaces are similar in that concrete objects from either one cannot be instantiated.
- Abstract classes differ from interfaces in that abstract classes can contain nonabstract methods, but all methods within an interface must be abstract.
- A class can inherit from only one base class (whether abstract or not), but it can implement any number of interfaces.

Example

- To create a Child that inherits from a Parent class and implements two interfaces, IWorkable and IPlayable,
 - A class must be defined with name and list the base class and interfaces separated by commas

```
class Child : Parent, IWorkable, IPlayable
```

- An existing interface needs to be implemented for a class to be able to use a method that already exists in other applications.
- For example,
 - Suppose a Payroll application is created that uses the Work() method in the interface class.
 - Also suppose a new class named BusDriver is created.
 - If BusDriver implements the Iworkable interface, then BusDriver objects can be used by the existing Payroll program.

- It is sometimes difficult to decide when to create an abstract base class and when to create an interface.
- Follow these guidelines:
 - Typically, an abstract class can be created when it is required to provide some data or methods that derived classes can inherit
 - When the subclasses needs to override some specific methods that is declare to be abstract.
 - An interface is created when derived classes require to override every method.
 - Use a base class when the class to be create “is a” subtype of another class; use an interface when the class to be create will act like the interface.

Polymorphism

- Polymorphism is often referred to as the third pillar of object-oriented programming, after encapsulation and inheritance.
- Polymorphism is a Greek word that means "many-shaped" and it has two distinct aspects:

- At run time, objects of a derived class may be treated as objects of a base class in places such as method parameters and collections or arrays.
- When this occurs, the object's declared type is no longer identical to its run-time type.

- Base classes may define and implement virtual methods, and derived classes can override them, which means they provide their own definition and implementation.
- At run-time, when client code calls the method, the CLR looks up the run-time type of the object, and invokes that override of the virtual method.
- Thus in the source code, a method on a base class can be called, and cause a derived class's version of the method to be executed.

- Virtual methods enable to work with groups of related objects in a uniform way.
 - For example, a drawing application that enables a user to create various kinds of shapes on a drawing surface.
- At compile time which specific types of shapes the user will create is not known.
- However, the application has to keep track of all the various types of shapes that are created, and it has to update them in response to user mouse actions.

- Polymorphism can be used to solve this problem in two basic steps:
- Create a class hierarchy in which each specific shape class derives from a common base class.
- Use a virtual method to invoke the appropriate method on any derived class through a single call to the base class method.

- First, create a base class called Shape, and derived classes such as Rectangle, Circle, and Triangle.
- Give the Shape class a virtual method called Draw, and override it in each derived class to draw the particular shape that the class represents.
- Create a List<Shape> object and add a Circle, Triangle and Rectangle to it.
- To update the drawing surface, use a [foreach](#) loop to iterate through the list and call the Draw method on each Shape object in the list.
- Even though each object in the list has a declared type of Shape, it is the run-time type (the overridden version of the method in each derived class) that will be invoked.

```
public class Shape
{ // A few example members
  public int X { get; private set; }
  public int Y { get; private set; }
  public int Height { get; set; }
  public int Width { get; set; }
  // Virtual method
  public virtual void Draw()
  {
    Console.WriteLine("Performing base class drawing
      tasks");
  }
}
```

```
class Circle : Shape
{
    public override void Draw()
    { // Code to draw a circle...
        Console.WriteLine("Drawing a circle");
        base.Draw();
    }
}
```



```
class Rectangle : Shape
{
    public override void Draw()
    { // Code to draw a rectangle...
        Console.WriteLine("Drawing a rectangle");
        base.Draw();
    }
}
```

```
class Triangle : Shape
{
    public override void Draw()
    {
        // Code to draw a triangle...
        Console.WriteLine("Drawing a triangle");
        base.Draw();
    }
}
```

```
class Program { static void Main(string[] args)
{
    // Polymorphism at work #1: a Rectangle, Triangle and Circle
    // can all be used wherever a Shape is expected. No cast is
    // required because an implicit conversion exists from a derived
    // class to its base class.
```

```
System.Collections.Generic.List<Shape> shapes = new
    System.Collections.Generic.List<Shape>();
shapes.Add(new Rectangle());
shapes.Add(new Triangle());
shapes.Add(new Circle());
```

```
// Polymorphism at work #2: the virtual method Draw is
// invoked on each of the derived classes, not the base class.
foreach (Shape s in shapes)
{
    s.Draw();
}
// Keep the console open in debug mode.
    Console.WriteLine("Press any key to exit.");
    Console.ReadKey();
}
}
```

/* Output:

Drawing a rectangle

Performing base class drawing tasks

Drawing a triangle

Performing base class drawing tasks

Drawing a circle Performing base class drawing
tasks */

Interface Polymorphism

- Same as regular polymorphism except that interfaces are used instead of abstract classes and methods.
- The result is also the same.

Type operators

- The *is* and *as* type operators convenient tools for testing the pedigree of an instance
- These operators confirm the presence of a class or interface in the hierarchy of an instance
- Syntax
 - `instance is type;`
 - The instance operator returns true if the instance is related (same type or is derived from the type) to the type, else false.

Attribute inheritance

- By default, custom attributes are not inherited
- ***AttributeUsage*** attribute can make an attribute inheritable.